

Web Avatar Lip-Sync

How a still image is made to "talk" in a browser · complete code with explanation · React / Next.js + CSS

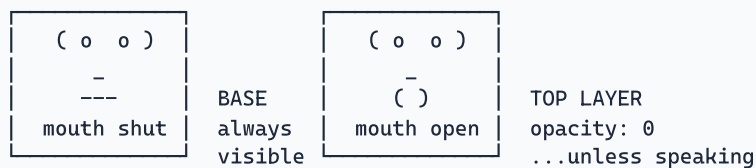
1. What this technique actually is

It is not real lip sync. There is no viseme model, no phoneme detection, and no audio analysis driving the mouth. It is a **two-image flip-book**: a mouth-closed image and a mouth-open image stacked in perfect registration, with the top one toggled on and off by a CSS `steps()` animation at roughly 3.6 flips per second. The animation is switched on and off by a single React boolean that tracks whether audio is currently playing. Three moving parts in total. It reads convincingly as a talking head at normal viewing distance while costing essentially nothing in CPU, bandwidth, or model weight.

The whole effect is about 55 lines of TSX, 12 lines of CSS, and two PNG files. Section 7 covers what to use instead if you need genuine phoneme-accurate lip sync.

The idea in one diagram

Two images, same box, pixel-registered:



When speaking, the TOP layer's opacity is snapped $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$ every 280ms by a CSS keyframe. Head, hair and shoulders are identical in both images, so the **ONLY** visible change is the mouth.

`audio.onplay` → `speaking = true` → animation class applied
`audio.onended` → `speaking = false` → class removed, mouth shut

Where it is accurate. Frame-exact at the *boundaries* — the mouth starts moving on the frame audio playback begins and stops on the frame it ends. Everything *between* those points is a fixed-rate loop with no relationship to the audio content. The 280 ms cycle was chosen because it sits near the natural syllable rate of conversational speech, which is what sells the illusion.

File map

Piece	File	Lines
Avatar component — stacks the two images	<code>app/page.tsx</code>	806–861
The flip animation + keyframes	<code>app/globals.css</code>	46–57, 101–104
Reduced-motion accessibility guard	<code>app/globals.css</code>	117–121
<code>speaking</code> state declaration	<code>app/page.tsx</code>	124
Audio playback — sets/clears the flag	<code>app/page.tsx</code>	342–374
Speech-synthesis fallback path	<code>app/page.tsx</code>	310–336
Teardown	<code>app/page.tsx</code>	162–167
Mic capture + voice-activity detection	<code>app/page.tsx</code>	185–305
Mouth-closed image	<code>public/avatar.png</code>	—
Mouth-open image	<code>public/avatar-speaking.png</code>	—

2. The avatar component

app/page.tsx · lines 806–861

Two Next.js `<Image>` elements with `fill`, inside the same `relative` circular container. Because both fill the identical box they are pixel-registered on top of each other — the head, hair and shoulders line up exactly, so when the top layer appears the *only* thing that visibly changes is the mouth.

```

function TalkingAvatar({
  speaking = false,
  size = "message",
}: {
  speaking?: boolean;
  size?: "message" | "welcome" | "stage" | "hero";
}) {
  return (
    <div
      className={cn(
        "relative flex-shrink-0 overflow-hidden rounded-full border-2 border-white",
        "shadow-md ring-1 ring-black/15",
        size === "hero"
          ? "h-56 w-56 sm:h-72 sm:w-72"
          : size === "welcome"
            ? "h-24 w-24 sm:h-32 sm:w-32"
            : size === "stage"
              ? "h-24 w-24 sm:h-32 sm:w-32"
              : "h-10 w-10",
        speaking && "avatar-speaking" // glow ring while talking
      )}
      aria-label={speaking ? "Avatar is speaking" : "Avatar"}
    >
      /* ---- BASE LAYER - mouth closed. Always visible. ---- */
      <Image
        src="/avatar.png"
        alt=""
        fill
        priority={size !== "message"}
        sizes={
          size === "hero"
            ? "(min-width: 640px) 288px, 224px"
            : size === "stage"
              ? "(min-width: 640px) 128px, 96px"
              : "40px"
          }
        className="avatar-face object-cover"
      />

      /* ---- TOP LAYER - mouth open. opacity:0 until animated on. ---- */
      <Image
        src="/avatar-speaking.png"
        alt=""
        fill
        priority={size !== "message"}
        sizes={
          size === "hero"
            ? "(min-width: 640px) 288px, 224px"
            : size === "stage"
              ? "(min-width: 640px) 128px, 96px"
              : "40px"
          }
        className={cn(
          "avatar-face object-cover opacity-0",
          speaking && "avatar-speaking-frame" // THE lip-sync class
        )}
      />
    </div>
  );
}

```

Why it is built this way

- `priority={size !== "message"}` — the large hero and stage avatars are preloaded. If the mouth-open image lazy-loaded, the first flip of every conversation would show a blank frame while it fetched. Small inline avatars stay lazy because there can be dozens on screen at once.
- **The `sizes` attribute** — tells the image optimiser exactly which responsive variant to serve per breakpoint. Without it the full-resolution original gets served into a 40 px circle.
- **`alt=""` on both images** — they are decorative duplicates of each other. The accessible name lives once on the wrapper via `aria-label`, which also announces speaking state to screen readers. Two non-empty alts would make a screen reader read the avatar twice.
- **The `speaking` prop is a plain boolean** — the component is pure and stateless. All timing lives in the parent, which makes the component trivially reusable and testable.

3. The CSS that does the actual animating

app/globals.css · lines 46–57 and 101–104

```
@layer components {
  .avatar-face {
    object-position: 50% 8%;      /* crop toward the face */
    transform: scale(1.7);        /* zoom so the mouth is large in the circle */
    transform-origin: 50% 12%;    /* zoom anchored at the head, not centre */
  }

  .avatar-speaking {
    box-shadow: 0 0 0 3px rgba(15, 23, 42, 0.12),
               0 8px 20px -8px rgba(15, 23, 42, 0.5);
  }

  .avatar-speaking-frame {
    animation: avatar-frame-talk 280ms steps(1, end) infinite;
  }
}

@keyframes avatar-frame-talk {
  0%, 42% { opacity: 0; } /* mouth CLOSED for ~118ms */
  43%, 100% { opacity: 1; } /* mouth OPEN for ~162ms */
}
```

Why `steps(1, end)` is the load-bearing detail

This single timing function is what separates a convincing flip-book from an obvious cross-fade. The default easing would smoothly interpolate opacity from 0 to 1 across the whole 280 ms — meaning that for most of every cycle you would be looking at a **50% blend of a closed mouth and an open mouth simultaneously**. That renders as a ghosted double-exposure and instantly reads as fake.

`steps(1, end)` forbids interpolation entirely. Opacity holds at its keyframe value and then *snaps*. You get two crisp, discrete animation cels — exactly how hand-drawn character animation has handled mouths for a century.

Value	Timing	Effect
280ms cycle	≈ 3.6 flips / sec	Near the natural syllable rate of conversational speech
0%–42% opacity 0	≈ 118 ms	Mouth closed — the shorter phase
43%–100% opacity 1	≈ 162 ms	Mouth open — deliberately longer; a mouth that spends more time open than shut reads as talking rather than chewing
steps(1, end)	—	No interpolation. Hard snap between two cels. Prevents the ghosting cross-fade
infinite	—	Loops for as long as the class is applied — duration is controlled by adding/removing the class in React, not by setting an animation length

Performance note

`opacity` is one of only two properties (with `transform`) that browsers can animate entirely on the compositor thread, without layout or paint. The flip therefore costs effectively nothing: no JavaScript runs per frame, and the main thread is never touched by the animation. This matters because the avatar animates *while* audio decodes and React re-renders the surrounding UI.

The accessibility guard

app/globals.css · lines 117–121

```
@media (prefers-reduced-motion: reduce) {  
  .avatar-speaking-frame {  
    animation: none;  
    opacity: 1;  
  }  
  .fade-up { animation: none; }  
  .ui-lift { transition: none; }  
}
```

A rapidly flickering face is a genuine problem for users with vestibular disorders or photosensitivity. Under reduced-motion the flipping stops, but `opacity: 1` keeps the mouth-open frame showing — so the avatar still visually indicates "I am speaking right now", it just does so statically instead of by flickering. The signal survives; the motion does not.

4. The React state that drives it

There is exactly one state value behind the whole effect:

```
// app/page.tsx : 124
const [speakingTurnTs, setSpeakingTurnTs] = useState<number | null>(null);
```

It stores the **timestamp of the message currently being spoken**, or `null` for silence. Storing a timestamp rather than a plain boolean is what lets multiple avatars on the page decide independently whether *they* are the one talking.

4a. Playing an audio clip and driving the mouth

app/page.tsx · lines 342–374

This receives base64 MP3 data, turns it into a playable Blob URL, and hooks playback lifecycle events to the `speaking` state. The two highlighted lines are the entire link between audio and animation.

```
// Play a base64 MP3. Falls back to browser speechSynthesis when
// audio.play() is blocked (autoplay policy, unsupported codec, decode error).
const playReply = (b64: string, turnTs: number, fallbackText: string) => {
  stopSpeaking();

  // Blob URL is more reliable than a data: URL - some browsers throttle or
  // reject large data URLs, and .decode().play() errors are cleaner.
  const bytes = Uint8Array.from(atob(b64), (c) => c.charCodeAt(0));
  const blob = new Blob([bytes], { type: "audio/mpeg" });
  const url = URL.createObjectURL(blob);
  const audio = new Audio(url);
  replyAudioRef.current = audio;

  audio.onplay = () => setSpeakingTurnTs(turnTs); // ► MOUTH STARTS MOVING

  const finish = () => {
    setSpeakingTurnTs(null); // ► MOUTH STOPS
    if (replyAudioRef.current === audio) replyAudioRef.current = null;
    URL.revokeObjectURL(url);
    if (shouldAutoListenRef.current) {
      window.setTimeout(() => startRecordingRef.current(), 250);
    }
  };

  audio.onended = finish;

  audio.onerror = (event) => {
    console.error("reply audio decode/playback error:", event, audio.error);
    URL.revokeObjectURL(url);
    speakResponse(fallbackText, turnTs);
  };

  void audio.play().catch((err) => {
    console.error("audio.play() rejected:", err);
    setVoiceDetail(`Audio blocked (${err?.name || err}); using browser voice.`);
    URL.revokeObjectURL(url);
    speakResponse(fallbackText, turnTs);
  });
};
```

Design notes worth calling out

- **Blob URL over data: URL** — a base64 `data:` URI for a 30-second MP3 runs to hundreds of kilobytes of URL string. Several browsers throttle or outright reject those, and playback errors surface as opaque failures. A Blob URL is a short handle to binary already in memory.
- **onplay, not "right after calling play()"** — `play()` returns a promise that resolves before the first audio frame is actually rendered. Hooking `onplay` means the mouth starts on the frame sound actually begins, not tens of milliseconds early. This is the single most important detail for the effect looking synced.
- **revokeObjectURL on every exit path** — end, error, and rejected play all revoke. Miss one and each clip leaks a multi-hundred-KB blob for the page's lifetime.
- **The identity check** `if (replyAudioRef.current === audio)` — guards a race. If a new clip starts while the previous one is still playing, the old `onended` would otherwise null out the *new* audio's ref.
- **Two independent fallbacks** — `onerror` covers decode failure; `.catch()` covers autoplay-policy rejection. Both keep the avatar speaking rather than freezing mid-sentence.
- **The 250 ms delay before re-listening** — restarting the microphone the instant playback ends risks the tail of the speaker's own audio bleeding into the next recording, causing the app to transcribe itself.

4b. The speech-synthesis fallback

app/page.tsx · lines 310–336

Used when no audio clip was available, or when MP3 playback failed. Note it drives the *identical* state variable, so the avatar behaves the same regardless of which engine is actually producing sound.

```
const speakResponse = (text: string, turnTs: number) => {
  if (mode !== "avatar" || !("speechSynthesis" in window)) return;

  stopSpeaking();

  // Strip anything that would be read aloud as noise.
  const cleanText = text
    .replace(/https?:\/\//g, "") // URLs
    .replace(/[*_#`]/g, "") // markdown punctuation
    .trim();
  if (!cleanText) return;

  const utterance = new SpeechSynthesisUtterance(cleanText);
  utterance.lang =
    ({ en: "en-IN", hi: "hi-IN", ta: "ta-IN", bn: "bn-IN" } as Record<string, string>)[
      lang
    ] || lang;
  utterance.rate = 0.95; // slightly slower - clearer for first-time listeners
  utterance.pitch = 1.05;

  utterance.onstart = () => setSpeakingTurnTs(turnTs);
  utterance.onend = () => {
    setSpeakingTurnTs(null);
    if (shouldAutoListenRef.current) {
      window.setTimeout(() => startRecordingRef.current(), 250);
    }
  };
  utterance.onerror = () => setSpeakingTurnTs(null);

  window.speechSynthesis.speak(utterance);
};
```

Why this is only a fallback. Browser `speechSynthesis` depends on voice packs installed on the user's operating system. Coverage for non-English languages is wildly inconsistent — on many Windows machines there is simply no voice for a given language, and the utterance either plays in an English voice mangling the text, or silently does nothing at all. Server-rendered audio avoids that entirely, which is why it is the primary path. The browser path exists so the feature degrades instead of dying.

4c. The teardown

app/page.tsx · lines 162–167

```
const stopSpeaking = () => {
  window.speechSynthesis?.cancel(); // kill browser TTS
  replyAudioRef.current?.pause(); // kill MP3 playback
  replyAudioRef.current = null;
  setSpeakingTurnTs(null); // close the mouth immediately
};
```

Called at the top of every new turn, on unmount, and before any new playback — so an interrupted clip never leaves the avatar flapping silently.

4d. Wiring the component up

```
// small avatar in the header
<TalkingAvatar speaking={speakingTurnTs !== null} />

// the large hero avatar
<TalkingAvatar size="hero" speaking={speakingTurnTs !== null} />

// per-message avatar inside the transcript
<TalkingAvatar speaking={t.role === "assistant" && t.ts === speakingTurnTs} />
```

That last line is why `speakingTurnTs` holds a timestamp rather than a boolean. The transcript may contain twenty message bubbles; comparing `t.ts === speakingTurnTs` means only the bubble whose audio is *actually playing right now* animates. Everything else stays still.

5. Microphone capture and voice-activity detection

app/page.tsx · lines 185–305

Not lip sync, but it is the other half of a voice-driven avatar and the only place here that genuinely analyses waveform data. The same `AnalyserNode` + RMS pattern is the basis of the upgrade in section 6, so it is worth reading closely.

5a. Starting the recorder

```
const startRecording = async () => {
  if (
    mode !== "avatar" ||
    busy ||
    speakingTurnTs !== null ||           // never record while she's talking
    // "error" is retryable - a denied mic prompt must not brick the button.
    (voiceStatus !== "ready" && voiceStatus !== "error")
  ) {
    return;
  }

  try {
    const stream =
      mediaStreamRef.current ||
      (await navigator.mediaDevices.getUserMedia({
        audio: {
          echoCancellation: true,
          noiseSuppression: true,
          autoGainControl: true,
        },
      }));
    mediaStreamRef.current = stream;
    shouldAutoListenRef.current = true;
    audioChunksRef.current = [];

    // Pick a mimeType the browser actually supports. Letting the browser
    // choose fails on some Chrome/Edge builds on Windows with "There was an
    // error starting the MediaRecorder" - explicit probing avoids that.
    const preferredMimes = [
      "audio/webm;codecs=opus",
      "audio/webm",
      "audio/ogg;codecs=opus",
      "audio/mp4",
    ];
    const supportedMime = preferredMimes.find(
      (m) => typeof MediaRecorder !== "undefined" && MediaRecorder.isTypeSupported(m)
    );
    const recorder = supportedMime
      ? new MediaRecorder(stream, { mimeType: supportedMime })
      : new MediaRecorder(stream);
    mediaRecorderRef.current = recorder;

    recorder.onerror = (event) => {
      const err = (event as unknown as { error?: DOMException }).error;
      console.error("MediaRecorder error:", err);
      setVoiceStatus("error");
      setVoiceDetail(err ? `${err.name}: ${err.message}` : "Recorder failed mid-capture");
    };

    recorder.ondataavailable = (event) => {
      if (event.data.size > 0) audioChunksRef.current.push(event.data);
    };

    recorder.onstop = async () => {
      try {
        const blob = new Blob(audioChunksRef.current, { type: recorder.mimeType });
        if (blob.size < 1200) {
          // Nothing meaningful captured (mic muted / stopped instantly).
          setVoiceStatus("ready");
          setVoiceDetail("I didn't catch any audio - tap the microphone and speak.");
          return;
        }
      }
    }
  }
}
```

```
    }  
    await sendVoice(await blobToBase64(blob));  
  } catch (cause) {  
    setVoiceStatus("ready");  
    setVoiceDetail(  
      cause instanceof Error  
        ? cause.message  
        : "Could not process microphone audio – tap the mic to try again"  
    );  
  }  
};  
  
recorder.start(250);           // emit a chunk every 250ms  
setVoiceStatus("recording");  
setVoiceDetail("Listening... speak naturally");
```

5b. The voice-activity detector — real waveform analysis

This is the auto-stop. It runs an `AnalyserNode` on a `requestAnimationFrame` loop, computing RMS energy of the raw time-domain samples about 60 times a second, and stops the recorder once the user has clearly finished speaking.

```
const captureContext = new AudioContext();
captureContextRef.current = captureContext;

const source = captureContext.createMediaStreamSource(stream);
const analyser = captureContext.createAnalyser();
analyser.fftSize = 1024;
source.connect(analyser);

const samples = new Uint8Array(analyser.fftSize);
const startedAt = performance.now();
let speechStarted = false;
let lastSpeechAt = startedAt;

const detectSilence = () => {
  analyser.getByteTimeDomainData(samples);

  // RMS energy: samples are 0-255 centred on 128 (silence).
  let energy = 0;
  for (const sample of samples) {
    const normalized = (sample - 128) / 128; // > -1.0 .. +1.0
    energy += normalized * normalized;
  }
  const rms = Math.sqrt(energy / samples.length);
  const now = performance.now();

  if (rms > 0.018) { // empirical speech-vs-room-noise threshold
    speechStarted = true;
    lastSpeechAt = now;
  }

  if (
    (speechStarted && now - lastSpeechAt > 1200) || // 1.2s silence AFTER speech
    (!speechStarted && now - startedAt > 10_000) || // 10s, never spoke at all
    now - startedAt > 30_000 // 30s hard ceiling
  ) {
    stopRecording();
    return;
  }

  vadFrameRef.current = requestAnimationFrame(detectSilence);
};

vadFrameRef.current = requestAnimationFrame(detectSilence);
} catch (cause) {
  setVoiceStatus("error");
  setVoiceDetail(
    cause instanceof Error ? cause.message : "Microphone access is required for voice mode"
  );
}
};
```

Constant	Value	Rationale
RMS threshold	0.018	Empirically above typical room tone and laptop fan noise, below normal conversational speech
Trailing silence	1200 ms	Long enough to survive a mid-sentence pause; short enough that the turnaround still feels responsive
No-speech timeout	10 s	Mic granted but nobody spoke — stop rather than record silence forever
Hard ceiling	30 s	Caps upload size and transcription cost even if someone talks continuously
Minimum blob	1200 bytes	Below this nothing meaningful was captured — don't waste a network round-trip
fftSize	1024	1024 time-domain samples per frame — plenty for an energy calculation, cheap enough for 60 fps

5c. Stopping cleanly

app/page.tsx · lines 169–183

```
const stopRecording = () => {
  if (vadFrameRef.current !== null) {
    cancelAnimationFrame(vadFrameRef.current); // stop the VAD loop
    vadFrameRef.current = null;
  }
  void captureContextRef.current?.close(); // release the AudioContext
  captureContextRef.current = null;

  const recorder = mediaRecorderRef.current;
  if (recorder?.state === "recording") {
    setVoiceStatus("processing");
    setVoiceDetail("Understanding what you said...");
    recorder.stop(); // fires onstop ▶ sendVoice()
  }
};
```

6. Middle ground — amplitude-driven mouth

Before reaching for a full lip-sync library, there is a cheap intermediate step that removes the most obvious tell of the flip-book: the fact that the mouth moves at a constant metronomic rate regardless of what is being said.

Map real-time audio loudness to how far the mouth opens. The mouth then genuinely follows the speech envelope — wide on stressed vowels, shut during pauses. It still will not match individual phonemes, but it stops looking mechanical, and it costs a few hours with no new dependencies.

The `AnalyserNode` + RMS pattern needed is exactly the one already written for the VAD in section 5b — it just points at the playback element instead of the microphone:

```
// Attach an analyser to the PLAYBACK audio instead of the mic
const ctx      = new AudioContext();
const source    = ctx.createMediaElementSource(audio); // the <audio> element
const analyser  = ctx.createAnalyser();
analyser.fftSize = 1024;
source.connect(analyser);
analyser.connect(ctx.destination); // MUST reconnect, or playback goes silent

const samples = new Uint8Array(analyser.fftSize);

const tick = () => {
  analyser.getTimeDomainData(samples);

  let energy = 0;
  for (const s of samples) {
    const n = (s - 128) / 128;
    energy += n * n;
  }
  const rms = Math.sqrt(energy / samples.length);

  // Drive opacity directly instead of running a fixed keyframe loop.
  setMouthOpen(Math.min(1, rms * 8)); // gain factor needs tuning by ear

  requestAnimationFrame(tick);
};
requestAnimationFrame(tick);
```

Then drop the `.avatar-speaking-frame` class and bind the top layer's opacity to that value directly.

Two gotchas that will cost you an hour each. First, `createMediaElementSource` reroutes the audio graph away from the speakers — forgetting `analyser.connect(ctx.destination)` silences playback entirely while everything else appears to work. Second, an element can only be passed to `createMediaElementSource` **once** for the lifetime of the page; calling it twice on the same element throws. The node must therefore be created alongside each new `Audio` object, not reused.

7. If you want proper lip sync — use a library

Everything above is a deliberate illusion. If you need mouth shapes that actually correspond to the sounds being spoken, do not hand-roll it — phoneme-to-viseme mapping and timing alignment is a solved problem with mature tooling. Pick based on what your avatar *is*.

Recommended: Rhubarb Lip Sync

Rhubarb Lip Sync · github.com/DanielSWolf/rhubarb-lip-sync · MIT, free, offline

This is the right answer for the two-image approach described in this document, because it targets exactly that model: 2D mouth *sprites*, not 3D rigs. You give it an audio file (and optionally the transcript, which sharply improves accuracy) and it returns a timed list of mouth shapes:

```
rhubarb -f json -d dialogue.txt speech.wav -o mouths.json

{"mouthCues": [
  { "start": 0.00, "end": 0.21, "value": "X" },    // rest / closed
  { "start": 0.21, "end": 0.34, "value": "B" },    // clenched / M,B,P
  { "start": 0.34, "end": 0.49, "value": "D" }     // open / AA
]}
```

You draw nine mouth images labelled **A–H** plus **X** (rest), then swap `src` as `audio.currentTime` crosses each cue boundary. The component structure in section 2 barely changes — you are replacing "toggle between 2 images on a timer" with "select from 9 images by timestamp". That is a small, well-defined diff over what already exists.

Integrating it — the shape of the change

```
// Instead of a CSS keyframe, select the shape by playback position.
const [shape, setShape] = useState("X");

const tick = () => {
  const t = audio.currentTime;
  const cue = mouthCues.find((c) => t >= c.start && t < c.end);
  setShape(cue?.value ?? "X");
  requestAnimationFrame(tick);
};

// ...and render the matching sprite
<Image src={`mouths/${shape}.png`} fill alt="" />
```

Alternatives, by use case

Library	Best for	Notes
Rhubarb Lip Sync MIT · offline	2D mouth sprites — the approach in this document	Pre-processes audio to a timed cue list. Nine mouth shapes. No runtime cost, no network, no API key. Requires a build step per audio clip, so it suits pre-recorded or server-generated audio better than live streaming.
wawa-lipsync MIT · browser	Real-time, in-browser, streaming audio	Analyses audio live in the browser via the Web Audio API and emits viseme weights frame by frame — no pre-processing pass. The right pick when audio arrives as a stream and you cannot run an offline analysis step first.
Azure Speech SDK commercial	You already generate speech with Azure TTS	Emits viseme events with precise offsets <i>during</i> synthesis, so alignment is exact by construction — the engine knows what it is about to say. Zero extra processing. Ties you to a paid cloud TTS.
Oculus / Meta OVR LipSync free SDK	Unity / Unreal, 3D characters	Mature, real-time, viseme output for blendshape-driven 3D face rigs. Not a web technology — irrelevant unless the avatar lives in a game engine.
TalkingHead (js) MIT	3D avatars on the web	Drives Ready Player Me / glTF avatars with visemes and expressions in three.js . Use if you want to move from a flat image to an actual 3D head.
Wav2Lip / SadTalker research · Python	Photorealistic video from a single photo	Generative models that take one still photograph plus an audio track and output a <i>video</i> of that face genuinely speaking it — the most convincing result available. Heavy: GPU inference, seconds-to-minutes per clip, so it is offline pre-rendering, not live conversation. Check the licence and, given they synthesise a real person's face, the consent and disclosure implications before shipping.

Choosing quickly. Pre-generated audio and a 2D avatar → **Rhubarb**. Live-streamed audio in the browser → **wawa-lipsync**. Already paying for Azure TTS → its built-in **viseme events**, which are free with the synthesis you are already doing. A 3D character → **TalkingHead** on the web, **OVR LipSync** in a game engine. Need it to look genuinely photoreal and can pre-render → **Wav2Lip**.

8. Summary

Question	Answer
Is it real lip sync?	No. Two-image flip-book, fixed rate, no phoneme or amplitude analysis.
What drives it?	One React value, set on <code>audio.onplay</code> and cleared on <code>audio.onended</code> .
What animates it?	A CSS <code>@keyframes</code> toggling opacity with <code>steps(1, end)</code> every 280 ms.
Total code involved	Roughly 55 lines of TSX, 12 lines of CSS, and 2 image files.
Runtime cost	Effectively zero — a compositor-thread opacity toggle. No JS runs per frame.
Where is it accurate?	At the start and end boundaries — frame-exact. In between, arbitrary.
Accessibility	<code>prefers-reduced-motion</code> stops the flicker but keeps the speaking indication.
Cheapest upgrade	Amplitude-driven opacity via <code>AnalyserNode</code> (\$6) — no new dependencies.
Proper solution	Rhubarb Lip Sync for 2D sprites; wawa-lipsync for live browser audio (\$7).

Web avatar lip-sync implementation walkthrough · line numbers reference the source file at time of writing.